

An Introduction to Recurrent Neural Networks

1. Limitations of Standard Neural Networks

Before introducing Recurrent Neural networks (RNNs), it is essential to understand the limitations of standard networks like Multi-Layer Perceptrons (MLPs) and Convolutional Neural Networks (CNNs). The primary problem is that standard neural networks are unable to handle sequential data effectively.

This limitation stems from two main issues:

1. First, standard networks treat each input independently, as if it stands alone. They lack a built-in mechanism to capture context across a sequence. In many real-world tasks, such as processing text, audio, or video, the order of information is crucial. Words in a sentence, for example, get their meaning from their surrounding words. The word "bank" is ambiguous on its own; its meaning becomes clear only from context, such as "financial bank" or "river bank". An MLP, having no memory of previous inputs, would struggle to resolve this ambiguity.
2. Second, standard neural networks require a fixed input size and a fixed output size. This is a significant problem because sequential data is often of variable length. For example, not all sentences have the same number of tokens. An MLP cannot naturally handle an input sentence of 5 tokens and another of 50 tokens.

2. Introduction to Recurrent Neural Networks (RNNs)

Recurrent Neural Networks (RNNs) were designed to overcome these limitations by specifically handling information that comes in a specific order, where earlier events influence what comes next. The core innovation of an RNN is its ability to remember past information by maintaining an internal memory, often called a hidden state.

Unlike an MLP, an RNN processes information step-by-step. At each time step t , it takes an input X_t and combines it with the information it remembered from the previous time step, H_{t-1} . This process updates its memory, creating a new hidden state H_t that captures the context from the beginning of the sequence up to that point.

This recurrent computation can be expressed with a generic equation:

$$H_t = f(H_{t-1} \cdot W_H + X_t \cdot W_E + B)$$

Here, H_t is the new hidden state (a vector), H_{t-1} is the previous hidden state, X_t is the current input, W_H and W_E are weight matrices, B is a bias, and f is a non-linear activation function. This dependency is what allows the network to preserve context across the sequence.

3. Core RNN Architecture and Principles

A key feature of the RNN architecture is that the weight matrices (W_H and W_E) are shared across all time steps. The hidden state vector evolves as new tokens are processed, but the

weights themselves remain constant. This weight-sharing provides a major advantage: symmetry in processing. In an RNN, because the same weight matrix is used repeatedly, every token is processed in the same way, giving it a fair chance to express itself. This also means the model size does not increase for longer input sequences.

This architecture is flexible and allows for optional outputs. The task determines whether an output is needed at every step:

- **Sequence Tagging:** For tasks like Part-of-Speech (POS) tagging, where every token needs a label, the model generates an output at every hidden state.
- **Sequence Classification:** For tasks like sentiment analysis, where only one label is needed for the entire sequence, the model processes all inputs and uses only the *final* hidden state to produce a single output.

4. Practical Applications and Language Modeling

Because RNNs excel wherever order matters, they are used in many everyday technologies.

- **Time Series Prediction:** They can predict trends like weather or stocks, or suggest the next song in a playlist based on listening history.
- **Sequence Recognition:** A smartwatch can use an RNN to recognize the pattern of your movements to classify whether you are walking, running, or cycling.
- **Natural Language Processing (NLP):** This is a major application. Predictive keyboards suggest the next word in a sentence, and voice assistants like Alexa or Siri process your commands in order to understand them.

A central application in NLP is building a language model, which is a model that understands and generates natural language. The task of a language model is to predict the next token that maximizes the probability of the entire sequence. To build this with an RNN, the input sequence is passed one token at a time, first by converting tokens into vectors using an embedding dictionary. The final hidden state is then passed through a feedforward (linear) layer and a Softmax function, which outputs a probability distribution over the entire vocabulary, allowing the model to pick the most likely next word.

5. Training RNNs and Key Challenges

Training Process:

To train a language model, we want it to generate the next best token at every time step. We can compare the model's prediction with the ground truth token from the dataset and compute an error³⁶. This error is calculated using cross-entropy loss (negative log likelihood), which forces the model to increase the probability of the correct token. The overall loss is the average of the losses at every time step.

During training, a technique called Teacher Forcing is often used. Instead of feeding the model's *own* (and potentially incorrect) generated output to the next time step, we "force" it by feeding the *correct* ground truth sequence as input. This stabilizes the training process.

Backpropagation Through Time (BPTT):

To minimize the loss, RNNs use an extended version of backpropagation called Backpropagation Through Time (BPTT). Because the loss at the end of the sequence depends on calculations from every previous time step, the algorithm must "unroll" the network through time and propagate the error signal backward from the end to the beginning.

Vanishing and Exploding Gradients:

This deep propagation through time leads to a major challenge. The gradient is calculated using the chain rule, which involves multiplying many derivative matrices together.

- **Vanishing Gradient Problem:** This is the most significant issue. If the eigenvalues of these matrices are small (less than 1), multiplying them together repeatedly causes the gradient to shrink exponentially, becoming "vanishingly" small. This means that the initial layers (early time steps) receive almost no gradient signal from the loss. As a result, their weights are not updated, and the model fails to learn long-term dependencies. For example, if a model needs to predict "tickets" at the end of a long paragraph, it may fail because it has "forgotten" the context of "tickets" mentioned at the very beginning.
- **Exploding Gradient Problem:** Conversely, if the eigenvalues are greater than 1, the gradient will grow exponentially and "explode". This is easier to solve; a common hack is gradient clipping, which sets a maximum threshold for the gradient's value.

6. Advanced Architectures: Bidirectional RNNs

A standard RNN only processes information from left-to-right, meaning its hidden state at any time t only contains information from the past. In many tasks, like translation or summarization, it's useful to know the *future* context as well.

A Bidirectional RNN (Bi-RNN) solves this by using two separate RNNs:

1. A forward RNN that reads the sequence from left-to-right (capturing past context).
2. A backward RNN that reads the sequence from right-to-left (capturing future context).

At any given time step t , the model generates two hidden states (one from each RNN), which are then typically concatenated. This gives the model a much richer understanding of the sequence by considering both past and future context for every token.